

¿Qué es un índice?

Un índice en bases de datos es una estructura de datos auxiliar que almacena una representación ordenada de los valores de uno o más campos de una colección, junto con punteros a los documentos originales.

Funciona de manera análoga al índice al final de un libro: en lugar de leer cada página para encontrar un término, se consulta el índice para obtener directamente la ubicación del dato.

Internamente, MongoDB implementa los índices como árboles B (B-tree), lo que permite búsquedas, inserciones y eliminaciones en tiempo $O(\log n)$ en lugar de $O(n)$ para un COLLSCAN (escaneo completo de la colección).

Ventajas de los índices

- Reducción drástica del tiempo de lectura (de $O(n)$ a $O(\log n)$)
- Evitan el COLLSCAN (escaneo completo) en colecciones grandes
- Soportan ordenamiento eficiente sin procesar todos los documentos
- Permiten enforced uniqueness (integridad referencial a nivel de campo)
- Habilitan búsqueda de texto completo con tokenización y stemming
- Mejoran el rendimiento de joins y lookups en pipelines de agregación
- Los índices geoespaciales permiten cálculos de proximidad en milisegundos

Desventajas de los índices

- Mayor consumo de espacio en disco (10%–40% del tamaño de la colección por índice)
- Ralentizan las operaciones de INSERT, UPDATE y DELETE (actualización del árbol)
- Mayor uso de RAM (el Working Set crece con cada índice adicional)
- Mantenimiento y fragmentación interna del árbol B con muchas escrituras
- Índices no utilizados penalizan rendimiento de escritura sin beneficio alguno
- Los índices de texto consumen considerablemente más recursos que los simples
- Crear índices en producción sobre colecciones grandes puede bloquear operaciones

Tipos de índices en MongoDB

MongoDB ofrece 9 tipos principales de índice, cada uno diseñado para optimizar un patrón de acceso específico:

Tipo	Descripción	Caso de uso
Single Field	Índice sobre un solo campo. Soporta orden ascendente (1) y descendente (-1).	Búsquedas simples por ciudad, nombre, ID
Compound	Hasta 32 campos. Regla ESR: Equality, Sort, Range para mayor eficiencia.	Filtros combinados: ciudad + rango de edad
Unique	Garantiza valores únicos. Puede ser sparse para admitir nulos.	Correos, números de cédula, usernames
Text	Tokenización y stemming para búsquedas full-text. Solo uno por colección.	Buscadores internos, campos de descripción
TTL	Elimina documentos automáticamente tras un período definido.	Logs de sesión, tokens de autenticación, caché
Geospatial (2dsphere)	Coordenadas GeoJSON. Queries de proximidad y dentro de polígonos.	Ubicación de domiciliarios, puntos de entrega
Hashed	Distribución uniforme en sharding. Solo igualdad, no rangos.	Fragmentación horizontal en clústeres
Wildcard	Indexa automáticamente todos los sub-campos de documentos dinámicos.	Colecciones con schema variable
Multikey	Se activa automáticamente cuando el campo indexado contiene un array.	Etiquetas, listas de habilidades, categorías

Casos reales — Uso empresarial

Los índices en MongoDB son fundamentales en plataformas de escala real. A continuación se presentan casos documentados:

E-commerce — Amazon / MercadoLibre

Utilizan índices compuestos en { categoría: 1, precio: 1, rating: -1 } para el motor de filtrado de productos. Sin índice, filtrar 50 millones de productos tomaría varios segundos; con el índice correcto, la respuesta cae a milisegundos. La regla ESR (Equality primero, luego Sort, luego Range) es crítica para maximizar la selectividad del índice compuesto.

Fintech — Nequi / Bancolombia

Usan índice único en número de cuenta y correo electrónico para garantizar que ningún cliente tenga dos cuentas asociadas. El índice actúa como constraint de integridad a nivel de base de datos, complementando las validaciones de la capa de aplicación. El error E11000 es capturado por el backend y devuelto como respuesta de negocio.

Sistemas de salud — HIS / EHR

Implementan índice TTL (Time-To-Live) sobre tokens de sesión de pacientes y profesionales de salud. Los registros de autenticación expiran automáticamente a las 8 horas sin necesidad de tareas programadas ni intervención del backend, reduciendo riesgos de seguridad y el tamaño de la colección de sesiones.

Logística — Rappi / Coordinadora

Usan índices geoespaciales 2dsphere sobre coordenadas de domiciliarios y puntos de entrega. Una query \$near sobre millones de ubicaciones devuelve el domiciliario más cercano en menos de 10 ms. Sin el índice geoespacial, esta operación requeriría calcular la distancia a cada documento: inviable a escala.

RRHH — Plataformas de empleo (LinkedIn, Computrabajo)

Aplican índice de texto en campos cargo, habilidades y descripción para búsqueda semántica de vacantes. El stemming del índice text permite que una búsqueda por "desarrolladora backend" también retorne documentos con "backend developer" o "desarrollo backend", mejorando el recall del buscador sin aumentar la complejidad de la query.

Impacto en rendimiento y almacenamiento

Beneficios en rendimiento de lectura

Con COLLSCAN, MongoDB lee todos los documentos de la colección para resolver la consulta (complejidad $O(n)$). Con un índice B-tree, la búsqueda es $O(\log n)$. Para una colección de 1 millón de documentos:

Estrategia	Documentos examinados	Tiempo estimado
COLLSCAN (sin índice)	~1,000,000	2,000 – 5,000 ms
IXSCAN (con índice)	~20 ($\log_2 1M$)	1 – 5 ms

Impacto en almacenamiento y escrituras

Cada índice adicional ocupa entre el 10% y el 40% del tamaño de la colección en disco. Los índices de texto son los más costosos porque mantienen la tabla invertida de tokens en memoria.

En cuanto a escrituras, MongoDB actualiza todos los índices en cada INSERT/UPDATE/DELETE. Con 4 índices (como en este taller), cada inserción realiza 4 actualizaciones internas del árbol B. En sistemas con alta frecuencia de escritura (100K+ ops/seg), esto puede representar un cuello de botella.